Handwritten Digit Recognition via Deep Neural Networks

1. Overview

My project focuses on the development and implementation mainly of the Multilayer Perceptron (MLP) model in AIML which is specially designed to classify the handwritten digits with high level precision. By using the **MNIST** dataset the model is trained to recognize patterns in images given by us. Beyond standard benchmarking, this project incorporates a custom inference pipeline using **OpenCV** to bridge the gap between static datasets and real world image processing for accurate predictions.

---

2. Problem Statement & Significance

The Challenge

Handwriting is inherently subjective to the nature of humans . Variations in slant, thickness, and style make it a complex task for traditional rule based rigit algorithms. The goal is to create a model that can generalize these variations of style and correctly identify digits from 0 to 9.

Why It Matters

While digit recognition is really important for:

- **Financial Automation:** Automated check processing and bank slip reading.
- **Postal Services:** Sorting mail via ZIP code recognition.
- **Historical Digitization:** Converting handwritten archives into searchable digital formats.

---

3. Data Engineering & Preprocessing

Data is the fuel of any given AI model. In this project, the popular **MNIST dataset** (70,000 samples) was utilized for the model.

Normalization

A critical decision was the use of tf.keras.utils.normalize in the model . The Raw pixel values range from 0 to 255. Feeding large integers into a neural network can lead to exploding gradients making training really unstable. By scaling these values to a range of

[0, 1], we make sure that the weight updates remain small in the model and the model converges faster.

Image Inversion for Real World Use Problem

MNIST data consists of white digits on a black background. However, in the real world handwriting is black on a white background. To solve this issue during the testing, I implemented the line np.invert() within the OpenCV pipeline to ensure the input data matched the training distribution for more accuracy.

---

4. Technical Approach & Architecture

The model was built using a **Sequential** architecture which is ideal for a linear stack of layers.

Layer Breakdown:

1. **Flatten Layer:** The input is a 28 times 28 matrix. The network cannot process 2D shapes directly in a Dense layer, so we unroll the pixels into a single vector of length 784.
2. **Hidden Layers :** Two layers of 128 neurons each. These layers act as the brain identifying increasingly complex features like edges, loops and intersections.
3. **Output Layer:** A 10 neuron layer using the Softmax activation. This converts the raw output scores into probabilities that sum to 1 .Example There is a 98% chance this is a 4.

Optimization Strategy

- **Optimizer (Adam):** An adaptive learning rate optimization algorithm that is generally even more efficient than standard Stochastic Gradient Descent.

---

5. Key Decisions & Observations

The Epoch 11 limit

During development, a key observation was made in the model training process regarding training duration. While more training usually improves performance the model was restricted to **3 epochs**.

- **Observation:** Beyond 11 epochs, the model risks Overfitting.
- **Impact:** Overfitting occurs when the model memorizes the random noises in the training set rather than learning the general features and proper given pattern.

This results in high accuracy on training data but a huge failure when presented with your own custom .png images for the model.

---

6. Challenges Faced

1. **Data Inversion:** Early tests did not perform well because the model expected white ink on a black background. Using the np.invert logic was what aligned the real world test images with the MNIST standard images.
2. **Normalizing Test Data:** Ensuring that the custom test images were normalized exactly like the training data was vital for maintaining a really good prediction consistency.

---

7. Lessons Learned & Future Work

Lessons Learned

- **Preprocessing is 90% of the Work:** The actual model architecture is simple but getting the images into the correct format is where the real engineering happens.
- **Simplicity vs. Power:** For a dataset as clean as MNIST a Dense MLP is highly effective 97% accuracy. Proving that you don't always need the most complex model to get really good results.

Future Enhancements

- **GUI Development:** Building a Tkinter interface where users can draw digits with their mouse.

---

8. Conclusion

This project successfully shows the power of TensorFlow and OpenCV in solving a classic classification AIML problem. By carefully balancing model depth with training time (epochs). I created a system that is functional in a real world testing environment to detect digits from people’s handwritten notes.